Unit-02
Introduction to MongoDB

❖ 2.1 Introduction of MongoDB

- NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of data.
- NoSQL databases use flexible data models that can adapt to changes in data structures and are capable of scaling horizontally to handle growing amounts of data.
-
- **Types of NoSQL database:**

Document databases:
These databases store data as semi-structured documents, such as JSON or XML, and can be queried using document-oriented query languages.
Examples – MongoDB, CouchDB, Cloudant

Key-value stores:
These databases store data as key-value pairs, and are optimized for simple and fast read/write operations.
Examples – Memcached, Redis, Coherence

Column-family stores:
These databases store data as column families, which are sets of columns that are treated as a single entity. They are optimized for fast and efficient querying of large amounts of data.
Examples – Hbase, Big Table, Accumulo

Graph databases:
These databases store data as nodes and edges, and are designed to handle complex relationships between data.
Examples – Amazon Neptune, Neo4j

- MongoDB is a widely-used, open-source NoSQL database that embraces a document-oriented data model. Unlike traditional relational databases, MongoDB stores data in flexible, JSON-like documents, allowing for dynamic schemas and easy scalability. It offers high performance, horizontal scalability, and is particularly well-suited for applications with rapidly evolving schemas and large amounts of unstructured or semi-structured data.

❖ 2.2 MongoDB features and advantages

There are many advantages of working with NoSQL databases such as MongoDB and Cassandra.
The main advantages are:

VPMP POLYTECHNIC

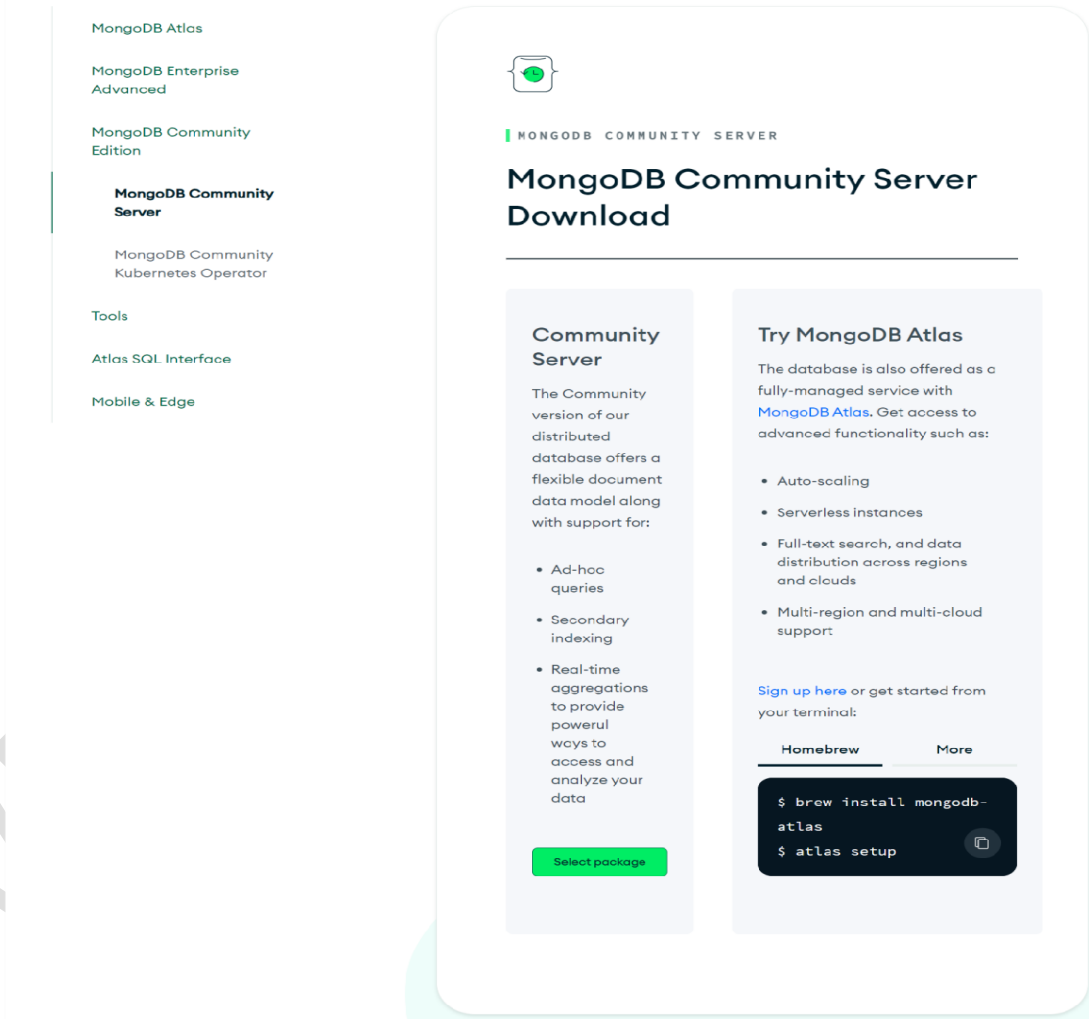
- Horizontal scalability
- high availability
- Flexibility
- Document-Oriented Data Model
- Dynamic Schema

.

❖ 2.3 Installing MongoDB

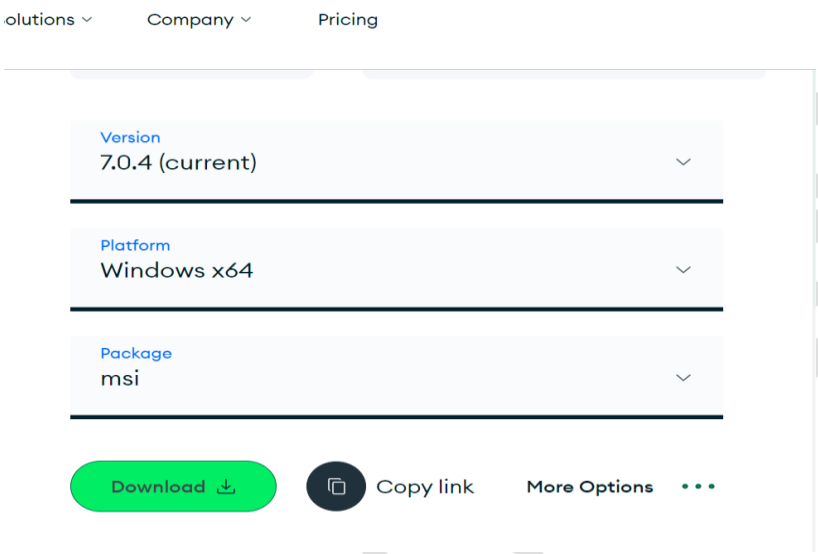
Downloading the MongoDB Installer:

- Visit the MongoDB website (https://www.mongodb.com/try/download/community) .

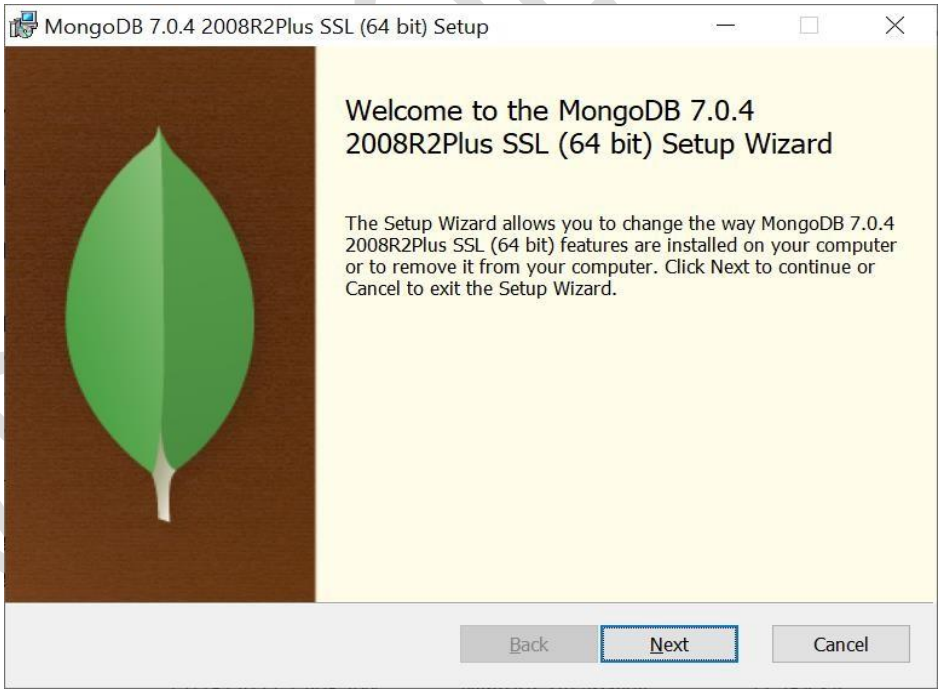


-

- Select the appropriate version for your operating system.
- Choose the desired edition (Community) and click the download button.
- Follow the prompts to save the installer file to your local machine.

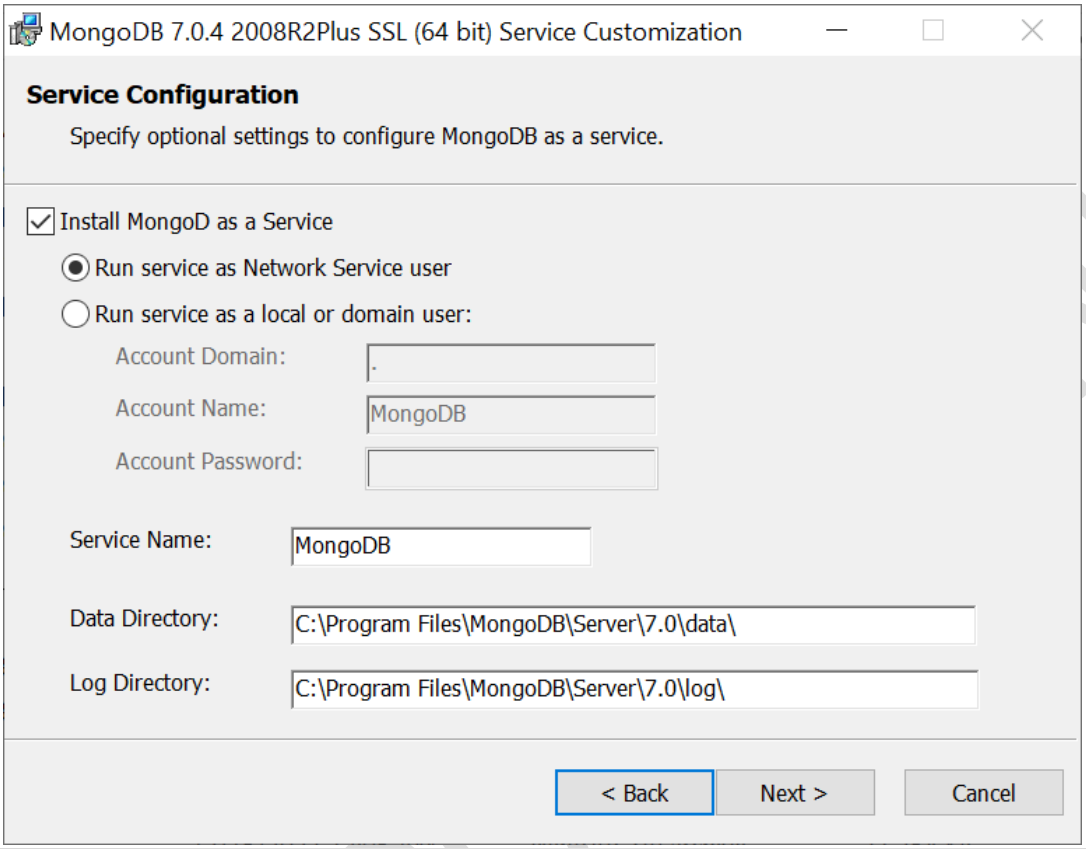


- Run the downloaded installer (.msi file).

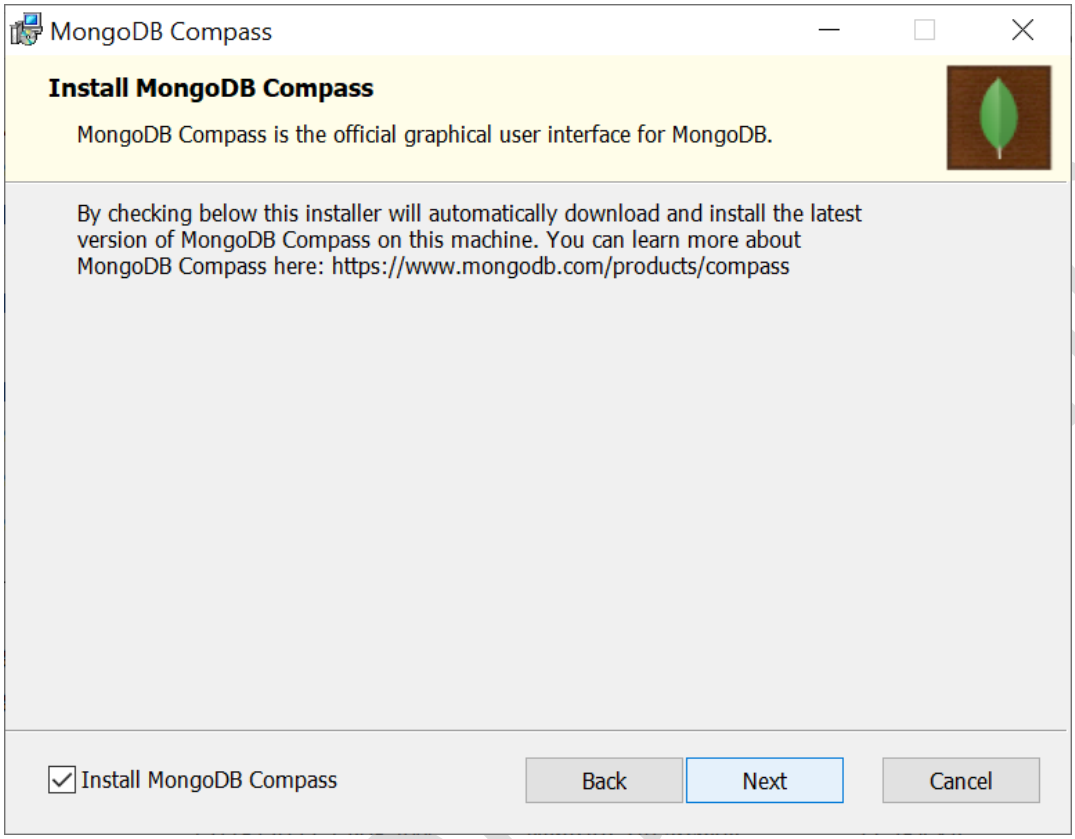


- Follow the installation wizard, accepting the license agreement.

- Choose the "Complete" setup type for a standard installation.



-MongoDB will be installed as a Windows service.



❖ 2.4 Connecting to MongoDB

MongoDB Connection String:

A MongoDB connection string is a URI-like string that specifies how to connect to a MongoDB instance. It includes information such as the host, port, authentication details, and other parameters. The general format is `mongodb://username:password@host:port/database`.

Connecting via MongoDB Shell:

1. Open a terminal or command prompt.
2. Run the `mongo` command.
3. Connect to a MongoDB instance using the connection string: `mongo "mongodb://username:password@host:port/database"`

❖ 2.5 Basic CRUD (Create, Read, Update, Delete) operations in MongoDB

1. The Create operation is used to insert new documents in the MongoDB database.

2. The Read operation is used to query a document in the database.
3. The Update operation is used to modify existing documents in the database.
4. The Delete operation is used to remove documents in the database.

Create Operations:

Create or insert operations add new documents to a collection.

If the collection does not currently exist, insert operations will create the collection.

MongoDB provides the following methods to insert documents into a collection:

- db.collection.insertOne()
- db.collection.insertMany()

Example:

```
db.products.insertOne({ name: "Laptop", price: 20000 })
```

Read Operations:

Read operations retrieve documents from a collection.

MongoDB provides the following methods to read documents from a collection:

- db.collection.find()

Examples:

```
db.products.find()
```

Update Operations:

Update operations modify existing documents in a collection.

MongoDB provides the following methods to update documents of a collection:

- db.collection.updateOne()
- db.collection.updateMany()
- db.collection.replaceOne()

Example:

```
db.products.updateOne({ name: " Laptop" }, { $set: { price: 30000 } })
```

VPMP POLYTECHNIC

Delete Operations:

Delete operations remove documents from a collection.

MongoDB provides the following methods to delete documents of a collection:

- db.collection.deleteOne()
- db.collection.deleteMany()

Example:

```
db.products.deleteOne({ name: "Laptop" })
```

❖ 2.6 Data modeling in MongoDB

In MongoDB, the way data is organized is very flexible. Unlike in traditional databases, you don't have to decide and declare how the data should look beforehand. Each piece of data (document) can be unique, but usually, they follow a similar pattern. The challenge is findinga good balance between what the application needs, how fast the database works, and howyou want to get the data back. When designing how data should look, think about how the data will be used in the application and how the data naturally fits together.

Document Structure in MongoDB:

Designing MongoDB data models involves crucial decisions on document structure and representing data relationships. Two approaches are commonly used: references andembedded documents.

❖ 2.7 Indexing and Query Optimization

Indexing is a fundamental optimization technique in MongoDB that significantly improves the efficiency of data retrieval operations. By creating indexes on specific fields, MongoDB accelerates query performance, enabling faster access to relevant documents within a collection.

1. Index Types:

- MongoDB supports various index types, including single-field, compound, multikey, and text indexes.
- Each index type caters to specific query patterns and enhances data access.

2. Index Creation:

VPMP POLYTECHNIC

- Indexes can be created using the `createIndex` method.
- Syntax: `db.collection_name.createIndex({ field_name: 1 })` for an ascending order index.

3. Default Index on `_id`:

- MongoDB automatically creates an index on the `_id` field for each collection.
- Additional indexes can be strategically created based on the application's query requirements.

Query optimization is a critical aspect of MongoDB database performance tuning, focusing on enhancing the efficiency of data retrieval operations. Optimized queries result in faster response times and improved overall performance.
Key Points:

1. Covered Queries:

- Aim for covered queries where the index alone satisfies the query without loading documents from the collection.
- Covered queries minimize the need to fetch documents, improving performance.

2. Query Execution Analysis:

- Utilize the `explain` method to analyze query execution plans and identify areas for optimization.
- Syntax: `db.collection_name.find({ field_name: "value" }).explain("executionStats")`.

3. Index Hinting:

- Use the `hint` method to explicitly specify which index MongoDB should use for a query.
- Syntax: `db.collection_name.find({ field_name: "value" }).hint({ field_name: 1 })`.

❖ 2.8 Aggregation Framework

Aggregation in MongoDB is a powerful framework that allows for the transformation and processing of documents within a collection. It enables users to perform advanced data manipulations, combining and transforming data in various ways to derive meaningful insights. Aggregation is especially useful for complex analytics and reporting tasks.

1. Pipeline Stages:

- Aggregation operations are structured as a pipeline, where each stage performs a specific transformation on the data.
- Common stages include `$match`, `$group`, `$project`, `$sort`, and `$unwind`.

VPMP POLYTECHNIC

2. Expression Operators:

- Aggregation expressions allow the manipulation of data within the pipeline stages.

-Examples include arithmetic operators (`$add`, `$subtract`), array operators (`$push`, `$addToSet`), and conditional operators (`$ifNull`, `$cond`).

3. Grouping and Summarization:

- The `$group` stage is fundamental for grouping documents based on specified criteria.

❖ 2.9 Replica Sets

A replica set in MongoDB is a distributed system that provides high availability and fault tolerance by maintaining multiple copies of data across multiple servers. This ensures data redundancy and allows for automatic failover in case of server failures. Replica sets are a fundamental component for building resilient MongoDB deployments.

❖ 2.10 Sharding in MongoDB

Sharding is a horizontal scaling technique in MongoDB designed to distribute data across multiple servers, or shards. This enables MongoDB to handle large datasets and high write and read traffic, ensuring scalability and improved performance. Sharding is a key feature for organizations with growing data demands.
Key Concepts:

1. Shard:

- A shard is an individual server or a replica set that stores a subset of the data.

- Shards collectively manage the entire dataset.

2. Shard Key:

- The shard key is a field chosen to distribute data across shards.

- Proper selection of the shard key is crucial for efficient sharding.

3. Balancer:

- MongoDB's balancer redistributes data across shards to maintain an even distribution.

VPMP POLYTECHNIC